

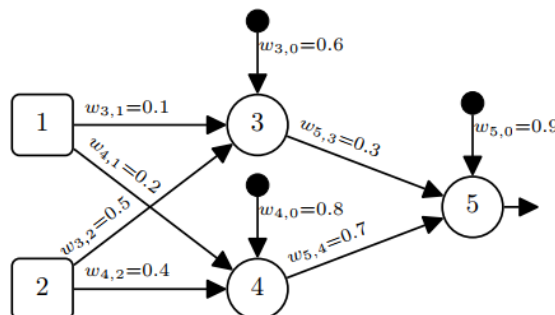
Machine Learning

Tutorial 6 — Deep Learning

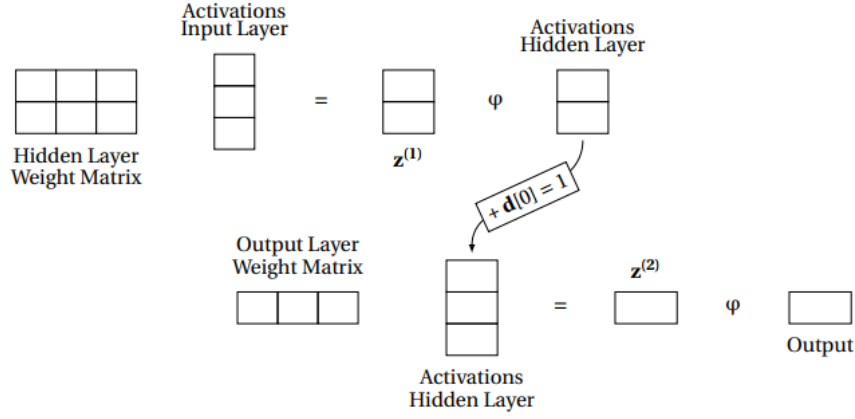
3rd Year · Semester 6 · 2025–2026

Exercise 1

The following image shows an artificial neural network with two sensing neurons (Neurons 1 and 2) and 3 processing neurons (Neurons 3, 4, and 5).

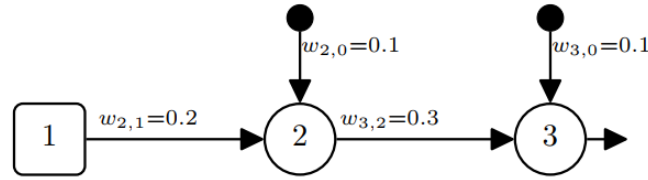


- (a) Assuming that the processing neurons in this network use a **logistic** activation function, what would be the output of Neuron 5 if the network received the input vector: Neuron 1 = 0.7 and Neuron 2 = 0.3?
- (b) Assuming that the processing neurons in this network use a **ReLU** activation function, what would be the output of Neuron 5 if the network received the input vector: Neuron 1 = 0.7 and Neuron 2 = 0.3?
- (c) The following image provides a template diagram for the sequence of matrix operations that our neural network would use to process the input vector Neuron 1 = 0.7 and Neuron 2 = 0.3. Assuming that the processing neurons in the network use a ReLU activation function, fill in the diagram with the appropriate weights, bias terms, weighted sum values, and activations.



Exercise 2

The following image illustrates the topology of a simple feedforward neural network that has a single sensing neuron (Neuron 1), a single hidden processing neuron (Neuron 2), and a single processing output neuron (Neuron 3).



(a) Assuming that the processing neurons use **logistic** activation functions, that the input to the network is Neuron 1 = 0.2 and that the desired output for this input is 0.7:

- Calculate the output generated by the network in response to this input.
- Calculate the δ values for each of the neurons in the network (i.e., δ_3, δ_2).
- Using the δ values you calculated above, calculate the sensitivity of the error of the network to changes in each of the weights of the network

$$\frac{\partial \mathcal{E}}{\partial w_{3,2}}, \quad \frac{\partial \mathcal{E}}{\partial w_{3,0}}, \quad \frac{\partial \mathcal{E}}{\partial w_{2,1}}, \quad \frac{\partial \mathcal{E}}{\partial w_{2,0}}.$$

- Assuming a learning rate of $\alpha = 0.1$, calculate the updated values for each of the weights in the network ($w_{3,2}, w_{3,0}, w_{2,1}, w_{2,0}$) after the processing of this single training example.
- Calculate the reduction in the error of the network for this example using the new weights, compared with using the original weights.

(b) Assuming that the processing neurons are **ReLU**s, that the input to the network is Neuron 1 = 0.2, and that the desired output for this input is 0.7:

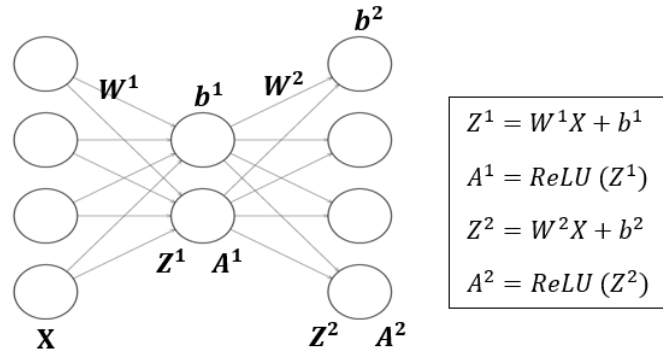
- Calculate the output generated by the network in response to this input.
- Calculate the δ values for each of the neurons in the network (i.e., δ_3, δ_2).
- Using the δ values you have calculated in the preceding, calculate the sensitivity of the error of the network to changes in each of the weights of the network

$$\frac{\partial \mathcal{E}}{\partial w_{3,2}}, \quad \frac{\partial \mathcal{E}}{\partial w_{3,0}}, \quad \frac{\partial \mathcal{E}}{\partial w_{2,1}}, \quad \frac{\partial \mathcal{E}}{\partial w_{2,0}}.$$

- iv. Assuming a learning rate of $\alpha = 0.1$, calculate the updated values for each of the weights in the network ($w_{3,2}$, $w_{3,0}$, $w_{2,1}$, $w_{2,0}$) after the processing of this single training example.
- v. Calculate the reduction in the error for this example using the new weights for the network, compared with using the original weights.

Exercise 3

In this exercise, our goal is to utilize an artificial neural network to perform vector compression, thereby reducing the storage requirements of a vector database. The neural network should compress the input vector without losing significant information and be capable of reconstructing the original vector when needed.



The ReLU (Rectified Linear Unit) activation function shown in the forward formulas is an element-wise ReLU, which means it applies standard ReLU on each element of the vector individually. The network architecture consists of two main components:

- **Encoder:** Responsible for compressing the input vector. It has the parameters W^1 and b^1 , where W^1 is a matrix of 2 rows and 4 columns, and b^1 is a vector of two elements.
- **Decoder:** Responsible for reconstructing the input vector from the compression. It has the parameters W^2 and b^2 , where W^2 is a matrix of 4 rows and 2 columns, and b^2 is a four-element vector.

The network is trained to minimize the reconstruction error for each example, given by the loss function:

$$\epsilon = \frac{1}{2} \sum_{i=1}^4 (X_i - A_i^2)^2$$

Q1. Give the formulas of individual deltas δ_1^2 , δ_2^2 , δ_3^2 , δ_4^2 , δ_1^1 , δ_2^1 .

To improve training efficiency, we will vectorize delta computations. In this vectorization, the element-wise product ($\odot$) and the ReLU derivative (ReLU') might be used. ReLU' produces a vector with 1 only at positions corresponding to strictly positive elements and zeros elsewhere.

Q2. Determine the vectorized formulas of δ^1 and δ^2 .

Due to space considerations, we will use the same parameters for the encoder and decoder. This means that the encoder weight matrix is the transpose of the decoder one:

$$W^1 = (W^2)^T$$

Q3. Under this constraint, determine the partial derivative of any individual weight $W_{i,j}^1$ without recalculating the deltas. Justify your answer.

Another issue is that the activation of the compressed vector is too high. To address this, we penalize high activations by adding the squared L_2 norm of the hidden layer activations to the loss function. The new loss function becomes:

$$\varepsilon = \frac{1}{2} \sum_{i=1}^4 (X_i - A_i^2)^2 + (A_1^1)^2 + (A_2^1)^2$$

Q4. Determine the partial derivative of an individual weight $W_{i,j}^1$ with this additional penalty term. In this question, don't consider the constraint given in Q3.

Exercise 4

Context. You are tasked with evaluating and interpreting the behavior of different deep neural network architectures in a controlled experiment. You train three models on the same classification dataset. All models achieve similar training accuracy ($\sim 99\%$), but their behaviors differ significantly during validation and analysis.

- **Model A:** Shallow and wide — 1 hidden layer with 10,000 neurons using ReLU.
- **Model B:** Deep and narrow — 10 hidden layers with 100 neurons each, using ReLU.
- **Model C:** Very deep — 50 hidden layers with 100 neurons each, using **tanh** as the activation function.

You observe that **Model A** performs well on training data, but its **validation accuracy drops after a few epochs**. Meanwhile, Model B, with less total capacity, maintains better validation performance.

Q1. Without directly invoking the term *overfitting*, describe what might be happening inside Model A.

Q2. Suggest at least two techniques or strategies that might help improve generalization in Model A, and justify why you would consider trying them in a particular order.

Despite having fewer total parameters, Model B generalizes better than Model A.

Q3. Reflect on why deeper models might provide more useful representations for generalization, even when shallower models have more capacity.

Q4. What does this suggest about the hierarchical nature of representations learned in deep neural networks?

Model C, with 50 tanh-activated layers, struggles during training. Specifically, you find that the **early layers barely update their weights**, while the final layers do.

Q5. Propose a possible explanation for this phenomenon, but avoid using technical terms like “vanishing gradient” or “exploding gradient”.

Q6. How would you explain this behavior to someone new to deep learning, using intuitive or real-world analogies?

Exercise 5

Assuming a fully connected feedforward network where all the neurons use a linear activation function (i.e. $a_i = z_i$) and with the following topology:

- (a) 100 neurons in the input layer
- (b) 5 hidden layers with 2,000 neurons in each layer
- (c) 10 neurons in the output layer

If all the inputs to the network have been standardized to have a mean value of 0 and a standard deviation of 1, and the initial weights for the network are sampled from a normal distribution with mean 0.0 and standard deviation of $\sigma = 0.01$, then:

- (a) Calculate the variance of the z values across the neurons in the first hidden layer in the first iteration of training.
- (b) Calculate the variance of the z values across the neurons in the last hidden layer in the first iteration of training.

Assuming that the variance of the δ s for the output layer is equal to 1:

- (c) Calculate the variance of the δ s across the neurons in the last hidden layer in the first iteration of training.
- (d) Calculate the variance of the δ s values across the neurons in the first hidden layer in the first iteration of training.
- (e) Is the training dynamic of this network stable, or is it suffering from vanishing or exploding gradients?

Exercise 6

You are given a feedforward neural network with the following architecture:

- Input Layer: m features
- Hidden Layer 1: N neurons, ReLU activation
- Hidden Layer 2: N neurons, ReLU activation
- Output Layer: O neurons (for classification into O classes), no activation

All layers are fully connected and include bias terms. Unless otherwise specified, assume precision FP32 (4 bytes per number) and batch size 1.

1. Compute the total memory (in bytes) required to store all weights and biases of the network.
2. Compute the memory (in bytes) needed to store all intermediate activations during a forward pass for a single input.
3. Could we optimize this forward required memory during inference? If yes, explain how.
4. Compute the memory needed to store all the required gradients during backpropagation for one input.

5. Gradient checkpointing consists of not saving some intermediate activations during the forward pass, and recomputing them during the backward pass when needed. How does gradient checkpointing help reduce memory usage during training, and what kind of overhead does it introduce?
6. Mixed-Precision Training is a technique that uses both 16-bit (half-precision) and 32-bit (single-precision) floating-point types during training to reduce memory usage and speed up computation, while maintaining model accuracy. How does Mixed-Precision help reduce memory usage during training, and what kind of overhead does it introduce?

Exercise 7

Assuming a network architecture that has four neurons in a **softmax** output layer. If the one-hot encoding of the target for the current training example is $\mathbf{t} = [0, 0, 1, 0]$ and the logits for the four neurons in the softmax output layer for this example are $[0, 0.5, 0.25, 0.75]$, then what is the δ value for each of the four neurons?

Exercise 8

Assuming a feedforward neural network that has 4 neurons in hidden layer k and that we are training this network using **inverted dropout** with $\rho = 0.5$. If the activations for the neurons in layer k are as follows: $[0.2, 0.4, 0.3, 0.1]$ and the **DropMask** for layer k is $[1, 0, 1, 0]$, calculate the activation vector that is actually propagated to layer $k+1$ after inverted dropout has been applied.